OS-Lab4 Report

周皓瑜 3230104564

November 25, 2025

1 分工

周皓瑜：Task1 至 Task4，以通过 test 为准

徐晟洋 (3230104220)：Task5 及后续调试（我有让他好好学习我写的部分，所以请不要认为他的工作量太少）

2 吐槽

感觉第一个能评测的点在 Task4 不太合适，因为这样分工会比较困难。如果按 task1-3 分的话，由于前面的正确性不能保证，没法让队友在自己（有极大可能出 bug）的代码基础上开发，所以感觉至少得做到 task4 才算告一段落，但这样分配工作量就不合理，感觉这里有优化空间。

事实上，由于在 Task4 之前没有运行用户态程序，Task2 修改 entry.S 结束后，其正确性是未知的，所以感觉一定要写到 Task4 才能开始较为完整的调试，调掉先前写出的 bug。

3 Part1

3.1 Task 1

3.1.1 补全 copy_pgd

```
        // 3. 移动到下一个页面
        current_va += PGSIZE;
        pa += PGSIZE;
    }
}

pgd_t *copy_pgd(pgd_t *old_pgd)
{
    /* Lab4 Task1 */
    pgd_t *new_pgd = (pgd_t *)alloc_page();
    memset(new_pgd, 0, PGSIZE);

    // 稍后free_pgd
    // actually 512 = PGSIZE/8

    for (uint64_t i = 0; i < 512; i++) {
        uint64_t pte = old_pgd[i];
        // add this, for L2 PTE
        new_pgd[i] = pte;

        if (!(pte & PTE_V)) {
            continue;
        }

        if (i >= VA2VPN2(VM_DIRECT_START) &&
            i <= VA2VPN2(VM_DIRECT_END)) {
            // kernel region, just share
            continue;
        }

        uint64_t *level1_table = (uint64_t *)PA2VA(PTE2PA(pte));
        // add this, for L1 table
        uint64_t *new_level1_table = (uint64_t *)alloc_page();

        for (uint64_t j = 0; j < 512; j++) {
            uint64_t level1_pte = level1_table[j];
            // add this, for L1 PTE
            new_level1_table[j] = level1_pte;

            if (!(level1_pte & PTE_V)) {
                continue;
            }

            uint64_t *level0_table =
                (uint64_t *)PA2VA(PTE2PA(level1_pte));
            // add this, for L0 table
            uint64_t *new_level0_table = (uint64_t *)alloc_page();

            for (uint64_t k = 0; k < 512; k++) {
                uint64_t level0_pte = level0_table[k];
                // add this, for L0 PTE
                new_level0_table[k] = level0_pte;

                if (!(level0_pte & PTE_V)) {
                    continue;
                }

                void *va = (void *)PA2VA(PTE2PA(level0_pte));
                // modify put page to get page
                get_page(va);
            }
            get_page(level0_table);
        }
        get_page(level1_table);
    }
    get_page(new_pgd);

    return new_pgd;
}

void free_pgd(pgd_t *pgtbl)
{
    for (uint64_t i = 0; i < 512; i++) {
        uint64_t pte = pgtbl[i];
        if (!(pte & PTE_V)) {
            continue;
        }

        if (i >= VA2VPN2(VM_DIRECT_START) &&
            i <= VA2VPN2(VM_DIRECT_END)) {
            // kernel region, do not free
            continue;
        }

        uint64_t *level1_table = (uint64_t *)PA2VA(PTE2PA(pte));

        for (uint64_t j = 0; j < 512; j++) {
            uint64_t level1_pte = level1_table[j];
            if (!(level1_pte & PTE_V)) {
                continue;
            }

            uint64_t *level0_table =
                (uint64_t *)PA2VA(PTE2PA(level1_pte));
```

Figure 1: copy_pgd

如上图，仿照提供的 free_pgd 完成 copy_pgd，因为两者逻辑几乎相同，区别在于 copy 需要获取新页并根据旧 pte 设置新的 pte，而 free 是在已有页中根据已有 pte 释放页。

3.1.2 修改 proc.c 加入页表相关处理

在 task_init 中加入如下代码设置根页表，作为初始进程，其 pgd 指向内核页表

```
// 直接读取satp值作为初始进程的根页表
pgd_t *p_swapper_pg_dir;
asm volatile("csrr %0, satp" : "=r"(p_swapper_pg_dir));
init_task->pgd = (void *)PA2VA(((long long int)p_swapper_pg_dir << 12LL));
```

Figure 2: init_task 设置根页表

在 copy_process 中加入如下代码完成页表的深拷贝，使得每个进程拥有独立的页表，从而跑在地址相同的虚拟空间，地址不同的物理空间。

new_task->pgd = copy_pgd(old->pgd);

在 release_task 中加入如下代码释放页表。注意需要在完成各个页的释放之后且在 PCB 释放之前。

free_pgd(task->pgd);

3.2 Task 2

3.2.1 维护 user_sp 和 kernel_sp 的生命周期

在 task_init 中，加入如下代码，直接用汇编读取当前栈指针设置 kernel_sp

asm volatile(”mv %0, sp” : ”=r”(init_task->kernel_sp));

在 copy_process 中，加入如下代码，设置新进程的 kernel_sp

new_task->kernel_sp = (void *)new_task->stack + PGSIZE;

在 release_task 中，加入如下代码尝试释放用户栈。

if(task->user_sp != NULL) free_pages(task->user_sp);

如此，完成两个 sp 及其指向栈空间的生命周期维护。

3.2.2 检查并修改 __traps

```
_traps:
    /* Lab4 Task2 */
    csrrw tp, sscratch, tp
    bnez tp, from_user # 非零说明为PCB地址，即来自用户态

from_kernel:
    csrr tp, sscratch # 立刻恢复tp
    j save_context

from_user:
    sd sp, TASK_USER_SP(tp) # 切换用户栈至内核栈
    ld sp, TASK_KERNEL_SP(tp)

save_context:
    /* Lab1 Task3 */
    # 保存上下文
    sd x2, PT_SP-PT_SIZE_ON_STACK(sp) # sp
    addi sp, sp, -PT_SIZE_ON_STACK
    sd x1, 0(sp)      # ra
    # sd x2, PT_SP(sp) # sp
    sd x3, 16(sp)     # gp
    # sd x4, 24(sp)     # tp
    sd x5, 32(sp)     # t0
```

Figure 3: traps_enter

如上图，在进入 traps 时使用 scrrw 读取 sscratch 判断是否来自用户态，若其值非 0 则说明该值为 PCB 地址，说明来自用户态，执行用户态逻辑，否则其值为 0，说明 tp 中保存的即为 PCB 地址，执行内核态逻辑。若来自内核态，csrrw 将 tp 写入了 sscratch 并读出了 0，此时立刻从 sscratch 读回 tp 并进入上下文保存；若来自用户态，此时读出的是正确的 tp，通过 tp 找到 PCB 并进行用户栈和内核栈的切换，即将用户栈指针保存至 PCB 中，并从 PCB 中读取内核栈指针，之后进入上下文保存阶段。

不论如何，该操作完成后 sscratch 中的值均为先前 tp 的值，而 tp 指向 PCB。

```
save_context:
    /* Lab1 Task3 */
    # 保存上下文
    sd x2, PT_SP-PT_SIZE_ON_STACK(sp) # sp
    addi sp, sp, -PT_SIZE_ON_STACK
    sd x1, 0(sp)      # ra
    # sd x2, PT_SP(sp) # sp        You, 17小时前 • F
    sd x3, 16(sp)     # gp
    # sd x4, 24(sp)     # tp
    sd x5, 32(sp)     # t0
    sd x6, 40(sp)     # t1
```

Figure 4: traps_save1

```

    csrr t0, sscratch
    sd t0, 24(sp)

    csrw sscratch, x0

    # 调用 trap_handler，传递pt_reg地址
    mv a0, sp

    call trap_handler
```

Figure 5: traps_save2

如上图，保存上下文时先存入 sp 值（其实也可以在 x1 之后存入，但对称起来好看一点），并且从 sscratch 中读取将保存的 tp 值（如上文提到的，此时 sscratch 中存储的是 trap 前的 tp 值），存储后向 sscratch 写入 0 表明此时在内核态。之后传递 pt_regs 地址，调用 trap 处理程序。

```
    ld x30, 232(sp)  # t5
    ld x31, 240(sp)  # t6
    addi sp, sp, PT_SIZE_ON_STACK

    ld x2, PT_SP-PT_SIZE_ON_STACK(sp) # sp

    sret
```

Figure 6: traps_load

如上图，恢复时最后恢复 sp，保证切换完栈后直接完成处理并返回，如果仍然在 x1 之后读取 sp，可能会读到 task3 start_threads 设置的用户栈的值，导致恢复上下文出现问题。

3.2.3 修改 trap_handler

```
void trap_handler(struct pt_regs *regs)
{
    switch (regs->scause) {
    /* Lab1 Task4 */
    case SCAUSE_SSI:
        printk("Supervisor software interrupt handled\n");
        do_ssip();  // 清除软件中断等待位
        break;
    case SCAUSE_STI:
        // 设置下一次时钟中断
        //printk("timer interrupt handled\n");
        do_stip();

        break;

    default:
        LOG_ERR("Unhandled trap: scause = %s (0x%" PRIx64
            "), sepc = 0x%" PRIx64 ", stval = 0x%" PRIx64 "\n",
            scause_str(regs->scause), regs->scause, regs->sepc,
            regs->stval);
        for (;;)
            ;
    }
    /* Lab2 Task4 */        You, 上个月 • 保存本地修改
    if (regs->scause == SCAUSE_STI) { // 时钟中断
        schedule();
    }
}
```

Figure 7: trap_handler

如上图，修改 trap_handler 使其仅接受 pt_regs 参数 regs，并从中取出先前设置的 sepc,stval,scause 等参数即可。

3.3 动手做 Trap 来源的判断

如果在 Trap 开头不使用 sscratch 判断 Traps 来源，那么来自用户态时还需要从 sscratch 中读取 PCB 地址，否则无法使用 task_struct 切换栈，明明可以直接用 sscratch 的值解决的问题，再使用 SPP 会显得冗余。并且，为了保证存储过程中 tp 的正确性，必然有将 tp 写入 sscratch 这一步，这就不得不用 csrrw 指令操作 sscratch 了。

非要使用 csrrw 是为了防止原来 tp 的值被覆盖，如果使用 csrr，那么 trap 前 tp 的值会被覆盖，使用 csrrw 则将 tp 的值存入了 sscratch，在上下文保存环节中从 sscratch 中读出原先的

tp 值再存入即可。

4 Part2

4.1 动手做查看 uapp.elf 的 Program Header

```
root@zju-os /z/code (lab4)# readelf -W -l kernel/user/uapp.elf

Elf file type is EXEC (Executable file)
Entry point 0x0
There are 4 program headers, starting at offset 64

Program Headers:
  Type           Offset   VirtAddr           PhysAddr           FileSiz  MemSiz   Flg Align
  RISCV_ATTRIBUT 0x01047d 0x0000000000000000 0x0000000000000000 0x000065 0x000000 R   0x1
  LOAD           0x001000 0x0000000000000000 0x0000000000000000 0x0027a4 0x0027a4 R E 0x1000
  LOAD           0x004000 0x0000000000003000 0x0000000000003000 0x003000 0x003000 RW  0x1000
  GNU_STACK      0x000000 0x0000000000000000 0x0000000000000000 0x000000 0x000000 RW  0x10

 Section to Segment mapping:
  Segment Sections...
   00     .riscv.attributes
   01     .text .rodata .srodata.xdigits .eh_frame
   02     .data
   03
```

Figure 8: uapp.elf

如图是 uapp.elf 的 Program Header，根据该信息，我们要将两个 LOAD 部分的内容 (分别是.text .rodata .srodata.xdigits .eh_frame 段和.data 段) 加载到虚拟内存的 0x0 和 0x3000 处，并给第一部分设置 RE 权限，给第二部分设置 RW 权限。

4.2 Task 3

4.2.1 补全 load_elf_binary 函数

```c
int load_elf_binary(void *file, struct task_struct *task)
{
    Elf64_Ehdr *ehdr = (Elf64_Ehdr *)file;

    /* Lab4 Task3 */

    Elf64_Phdr *phdr = (Elf64_Phdr *)(file + ehdr->e_phoff);

    for(uint64_t i = 0; i < ehdr->e_phnum; i++)
    {
        // Elf64_Phdr *nw_phdr = (Elf64_Phdr *)((char *)phdr + i * ehdr->e_phentsize);
        Elf64_Phdr *nw_phdr = &phdr[i];
        for(uint64_t j = 0; j < nw_phdr->p_memsz; j += PGSIZE)
        //其实这里应该要用p_align的，不过p_align一般为PGSIZE，出问题了再修吧
        {
            uint64_t *page = (uint64_t *)alloc_page();
            memset(page, 0, PGSIZE);
            memcpy(page, file + nw_phdr->p_offset + j, PGSIZE);
            create_mapping(task->pgd, (void *)(nw_phdr->p_vaddr + j), (void *)VA2PA(page), \
            PGSIZE, flags_phdr_to_pte(nw_phdr->p_flags) | PTE_U);

            // LOG_DEBUG("create_mapping: %p, %p, %p\n",task->pgd, (void *)(nw_phdr->p_vaddr +
        }       You, 18小时前 • Finished to Task 4 (later will add more comments)…
    }

    // user stack
    void *user_stack = alloc_page();
    create_mapping(/* Lab4 Task3 */ task->pgd, (void *)VM_USER_STACK_TOP - PGSIZE,\
     (void *)VA2PA(user_stack), PGSIZE, PTE_R | PTE_W | PTE_U);

    // LOG_DEBUG("create_mapping: %p, %p, %p\n",task->pgd, (void *)VM_USER_STACK_TOP - PGSIZE,

    start_thread(/* Lab4 Task3 */ task_pt_regs(task), ehdr->e_entry, VM_USER_STACK_TOP);

    LOG_DEBUG("ELF loaded: entry = %p, PID = %d\n", (void *)ehdr->e_entry,
            task->pid);
    return 0;
}
```

Figure 9: load_elf_binary

如图是补充后的 load_elf_binary 函数，首先根据 ELF header 指针以及其中的 phoff 项定位到第一个 Program Header，然后根据 ehdr(ELF header) 中的 phnum 项遍历每个 phdr(Program Header)。

按照 phdr 中提供的 memsz 信息申请相应数量的内存，并使用 memcpy 从文件头 +offset 处读取段内容加载至申请的内存中，并使用 create_mapping 创造映射，从 phdr 中读取虚拟地址，映射至新 page 的物理地址，并设置 phdr 中规定的权限加上 U 位。

之后创建用户栈，将其虚拟地址设为用户空间的末尾，映射至物理地址，并设置权限为 RW，包含 U 位。

之后通过 start_thread 设置初始上下文，从 task 中获取上下文地址传入参数，将 PC 设为 ehdr 中的 entry 项，将栈指针设为用户栈栈顶，这样启动用户态程序时最终就可切换到用户栈，并进入用户态虚拟空间。

4.3 Task 4

4.3.1 补全 user_mode_thread

```c
struct task_struct *user_mode_thread(void *file)
{
    /* Lab4 Task4 */
    struct task_struct *new_task = copy_process(current);

    load_elf_binary(file, new_task);

    new_task->thread.ra = (uint64_t)ret_from_trap;
    new_task->thread.sp = (uint64_t)task_pt_regs(new_task);
    return new_task;
}
```

Figure 10: user_mode_thread

如上图补全 user_mode_thread，调用 load_elf_binary 将 ELF 文件加载至新的 task，并设置返回地址为 ret_from_trap 准备进入用户态，设置 sp 为 task 3 中 start_thread 里设置的初始上下文位置，完成用户态进程的配置。

References